



Dashboard Prepare ▾

New questions added in Oct 2026 →



Guide Menu

On this page

System design

Photo Sharing (e.g. Instagram)

★ Premium



Medium



30 mins



873 done

Report an issue

Asked at these companies



Airbnb



Dropbox



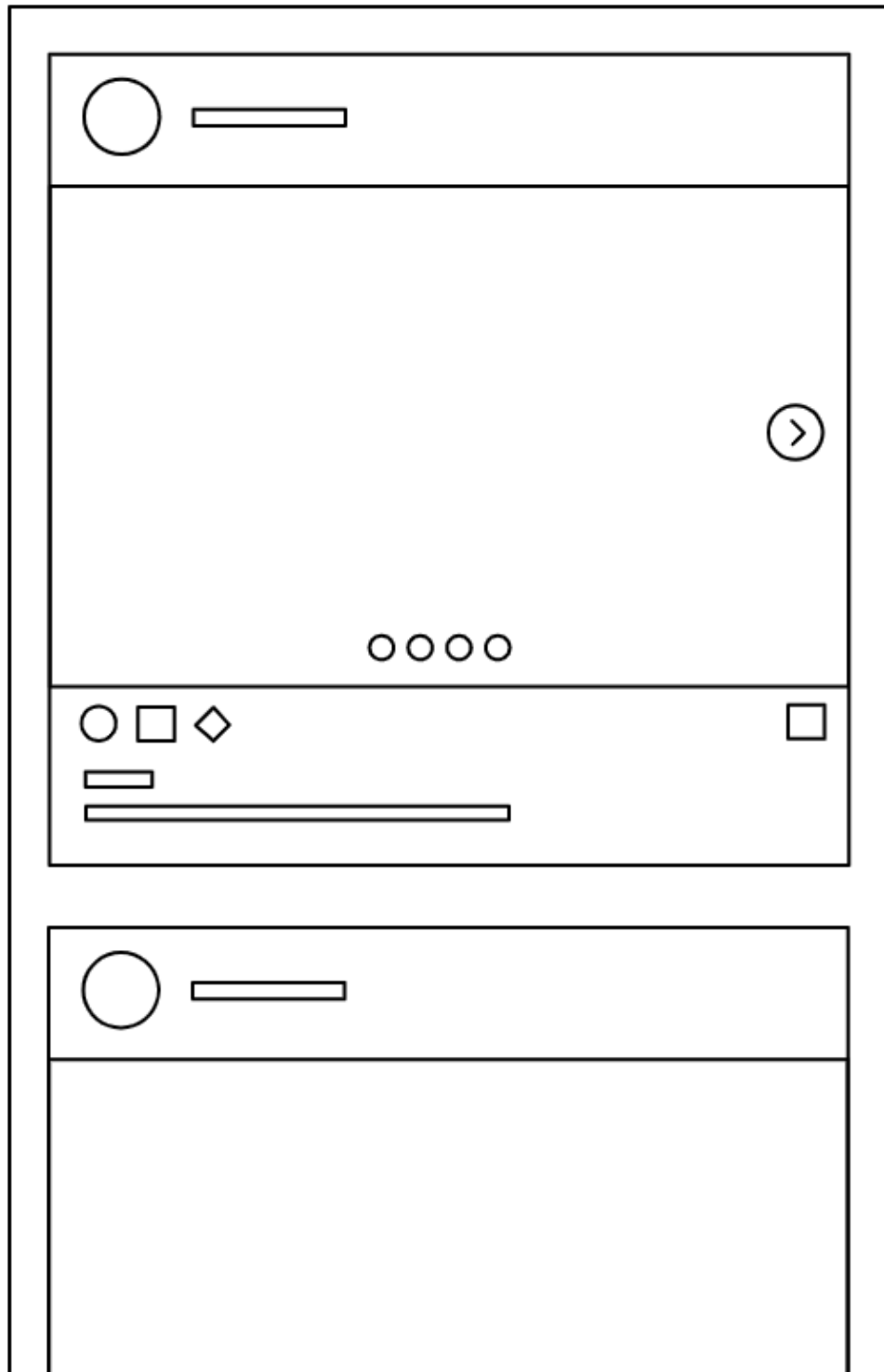
Meta



Apple

Question

Design a photo sharing application that contains a list of photo posts created by users. Users can create new posts containing photos. Public posts must also be viewable by logged-out users and discoverable by search engines.



Real-life examples

- <https://www.instagram.com>
- <https://www.flickr.com>

i Related: News Feed and Image Carousel

This application shares many similarities with the News Feed system design application, and the image carousels within the feed posts will benefit from the Image Carousel system design. Those questions are optional deeper reading rather than prerequisites. For this question, we will focus on things that haven't been covered, and the discussion will be centered around the photos and images.

★ Treat this article as the photo-specific delta on News Feed

The feed shape, pagination, rendering strategy, and store design are inherited from the News Feed write-up, and the in-post media UI is inherited from the Image Carousel write-up. In an interview, anchor on those defaults and spend your time on what is actually Instagram-specific: the image pipeline, upload flow, in-browser editing, and media-heavy rendering concerns.

Requirements exploration

We scope the exercise by clarifying which user journeys are in play (browsing, uploading, editing), which pagination pattern the feed uses, and which image-specific concerns belong in the interview.

What are the core features to be supported?

- Browse a feed containing image posts by people a user follows.
- View individual public posts without signing in.
- Upload photos, add captions, and apply filters to them before posting.

What pagination UX should be used for the feed?

Infinite scrolling, meaning more posts will be added when the user reaches the end of their feed.

Is the user able to add captions to individual photos/images or only one overall caption to the post?

Only one overall caption to the post.

What devices will the application be used on?

Primarily mobile, but should be usable on desktop and tablet as well.

Exercise Beta

You are asked to design a front-end photo-sharing experience centered on browsing and creating image posts. Which requirements should be included before choosing the architecture? Select all that apply.

- ☐ 1 Include comments, direct messages, follower-graph updates, and recommendation ranking in the baseline because each can affect photo discovery.
- ☐ 2 Use one caption for the overall post and allow filters to be applied before upload.
- ☐ 3 Use infinite scrolling for continuation and design primarily for mobile while keeping desktop and tablet usable.
- ☐ 4 Cover browsing the followed-photo feed, opening a public post, and creating a post by selecting, filtering, and captioning photos.



Reveal answer

Check answer ↩

Architecture / high-level design

Because posts must be reachable by logged-out users and crawlable by search engines, the architecture depends on the rendering model and how the client talks to the image and feed APIs.

Rendering approach

Photo sharing applications have the following characteristics:

- Posts are viewable by non-logged-in users.
- Posts are searchable via search engines.
- The application is interaction-heavy due to post composing and liking/commenting functionality on posts.
- Fast initial loading speed is desired.

These are also characteristics of a **News Feed system design**, where there's a good amount of static content that also requires interaction. Hence, server-rendering public post pages with hydration and using client-side rendering for later interactions would be ideal.

Architecture diagram

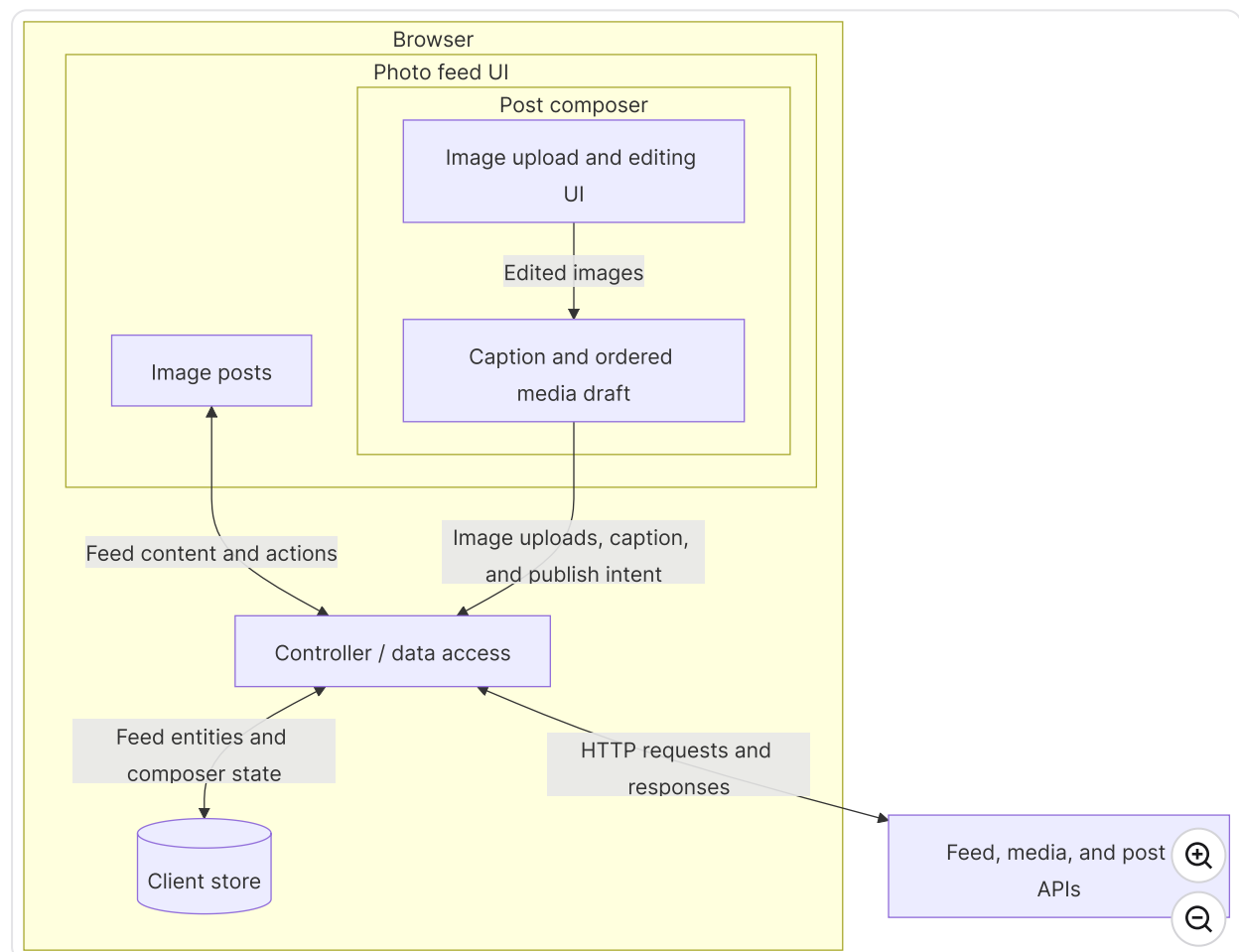


Photo-sharing client architecture

- **Server:** Provides HTTP APIs to fetch feed posts, upload images, and to create new image posts.
- **Controller:** Controls the flow of data within the application and makes network requests to the server.

- **Client store:** Stores data needed across the whole application. In the context of a photo sharing application, most of the data in the store will be server-originated data needed by the feed UI.
- **Feed UI:** Contains a list of image posts and the UI for creating new posts.
 - **Image post:** A post containing a list of one or more images.
 - **Post composer:** UI for uploading images, applying filters to them, and adding a caption before submitting to the server.

Exercise 1 of 2 Beta

Public photo posts need crawlable content and fast entry, while feed and composer interactions should remain responsive after load. Which rendering boundary best fits?

- 1 Server-render title and preview metadata, but leave the public post body empty until client JavaScript fetches it.
- 2 Server-render public post entry pages with their visible content and metadata, then hydrate client-side feed, carousel, and composer interactions.
- 3 Prebuild one generic photo feed and refresh it only at deployment time, then let clients upload posts into local state.
- 4 Server-render every feed continuation, carousel change, and composer step as a new document.

[Reveal answer](#)[Check answer ↩](#)

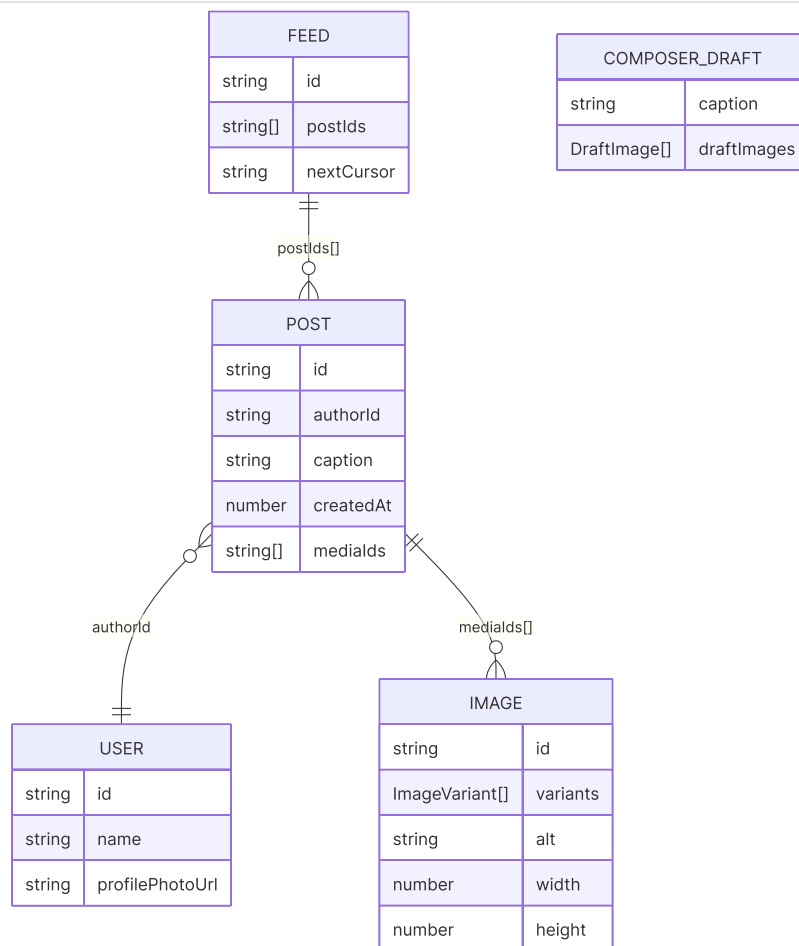
Data model

A photo feed shows image posts fetched from the server, so most durable data is server-originated. The client additionally keeps the active feed-request state and the user's in-progress images and caption in the post composer.

Entity	Source	Belongs to	Fields
Feed	Server	Feed UI	ordered postIds and cursor
Post	Server	Feed Post	id , created_time , caption , authorId , ordered mediaIds
User	Server	Client store	id , name , profile_photo_url
ComposerDraft	User input (Client)	Post composer UI	caption and ordered draft images, each with its local source, edit parameters, upload state, and eventual mediaId
Image	Server	Multiple	stable id , responsive variants , alt , width , height

Just like in the News Feed system design, a normalized client-side store will also be useful for a Photo Sharing application.

The Feed UI also keeps transient request state—its request key, generation, and status—without treating that state as part of the server-originated Feed entity.



Normalized client store for a photo feed with composer drafts

i Ship width and height with every image entity, not just the URL

Knowing the intrinsic dimensions up front lets the client reserve the correct aspect-ratio box before the image decodes, which is the main defense against cumulative layout shift in an image-heavy feed. It also lets the server or client pick the right srcset candidate without waiting for the bytes to arrive.

Exercise Beta

Which data and state boundaries keep reusable server entities separate from composer workflow and feed-request state? Select all that apply.

- 1 Treat the current canvas pixels as the edit source and recreate composer state from the mounted canvas when needed. ☐
- 2 Keep the active request key, generation, and status outside the server-originated Feed entity. ☐
- 3 Store posts by ID and reference ordered media records with dimensions and variants. ☐

4

Embed a mutable author and full post snapshot in each feed item so the view can render without entity lookups.

☐

5

Represent local media, upload progress, retry state, and server identity explicitly.

☐

Reveal answer

Check answer ↩

Interface definition (API)

The client talks to four API surfaces: the feed list, public post details, image uploads, and post creation.

Feed list API

Like News Feed's list API, the photo sharing application's feed list API should also use a **cursor-based pagination approach**.

Return ordered post IDs and normalized entities with an opaque next cursor. As part of its request state, the client captures the current feed generation when starting a fetch. It merges by stable post ID and ignores a response when its request key or captured generation no longer matches the active feed.

Public post API

`GET /posts/{postId}` returns only anonymously visible post, author, and media data. A stable public URL is server-rendered with a unique title, description, canonical URL, crawlable links, and image alternatives; private or missing posts return the appropriate non-indexable `404` /authorization response.

Post creation API

All posts in a photo sharing application have attached images. Hence, the post creation process in such apps is more complex.

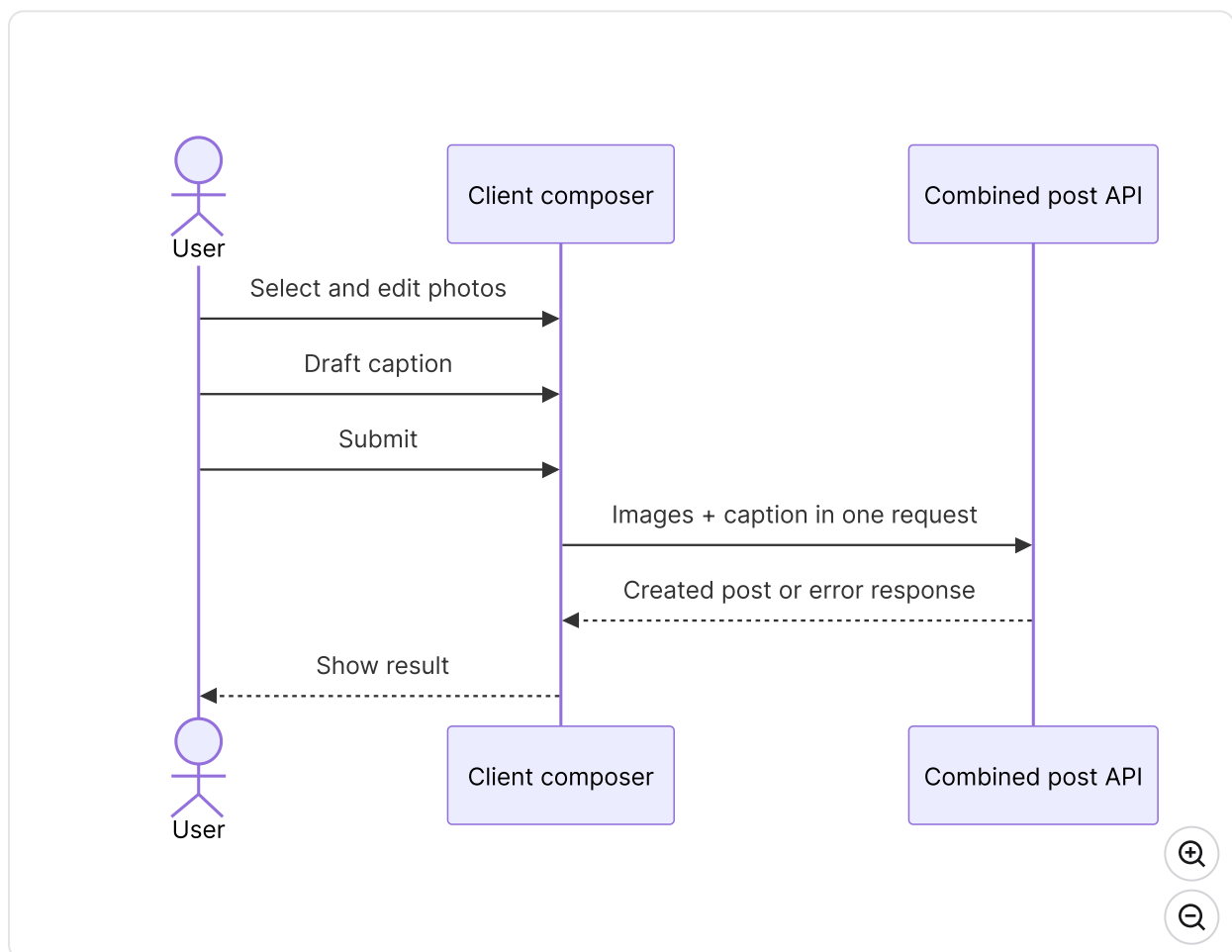
Let's break down the post creation into steps:

1. User selects photos from their device.
2. User is able to perform light edits on their photos: resize/crop/filter.
3. User adds a caption to the post.
4. User submits post.

There are two common ways to implement post creation functionality involving images:

1. Create a single API for both image uploading and post creation.
2. Create separate APIs for image uploading and post creation.

1. Create a single API for both image uploading and post creation



One request for media upload and post creation

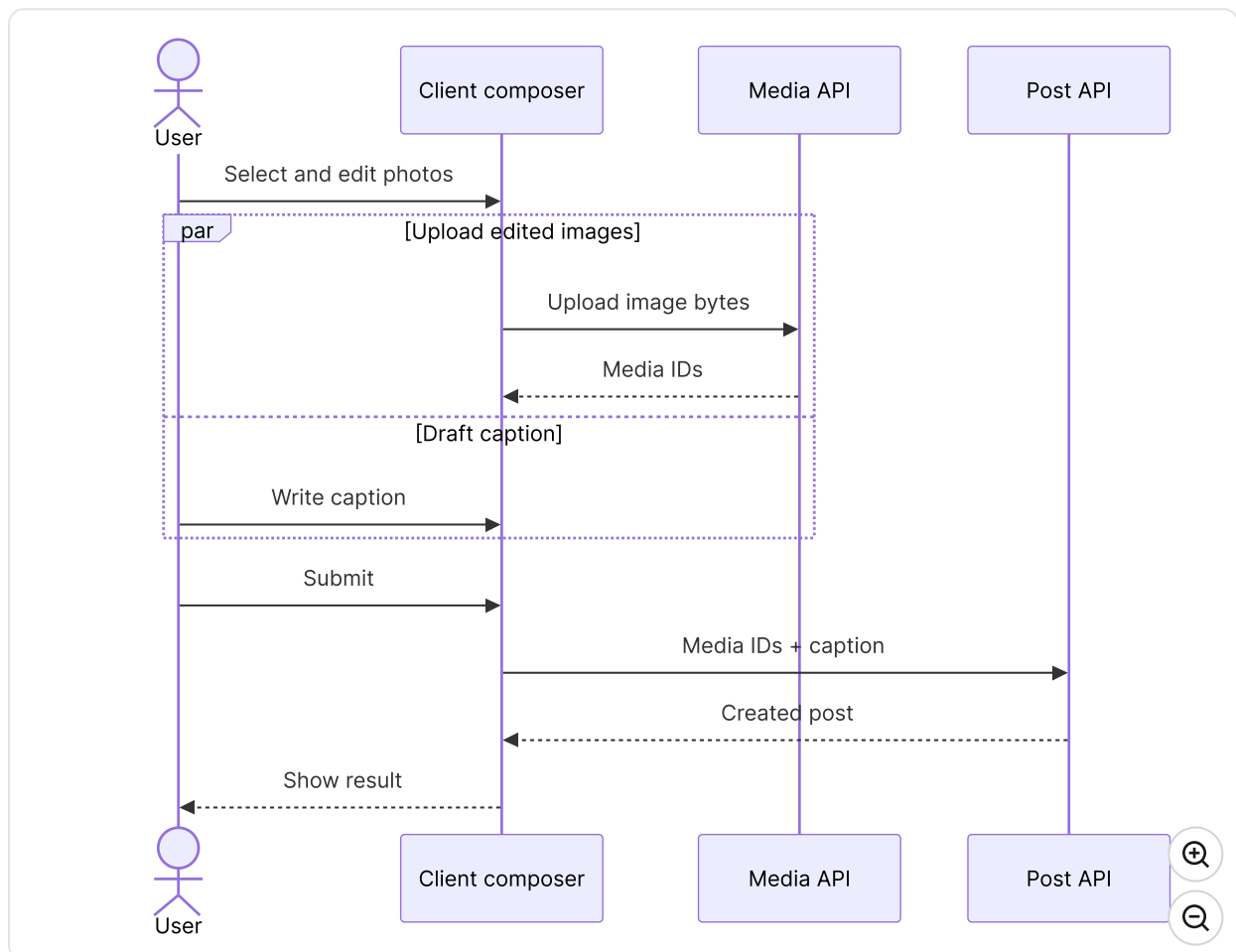
Pros:

- Simple to implement, upload all required data in one request.

Cons:

- API logic is more complex because it has to do multiple things on the back end (though that's not really a concern of the front end).
- Upload will take longer because the post images have to be uploaded within one request.
- The API has to take in both text-based data and media data.

2. Create separate APIs for image uploading and post creation



Separate media uploads and post creation

Pros:

- The image upload stage can be done in an async fashion. Images can be uploaded in the background after the image editing stage while the user is drafting the post caption.
- A small number of images can be uploaded concurrently across multiple HTTP requests, which is faster than uploading all the images in a single HTTP request without creating unbounded network work.
- Can leverage existing general image uploading APIs, if any.

Cons:

- More coordination is needed on the client. The client needs to wait for all the images to be uploaded, get references to the uploaded images, and use them when creating the post.

Which approach is better?

Comparing the two approaches, the separate APIs approach would be better.

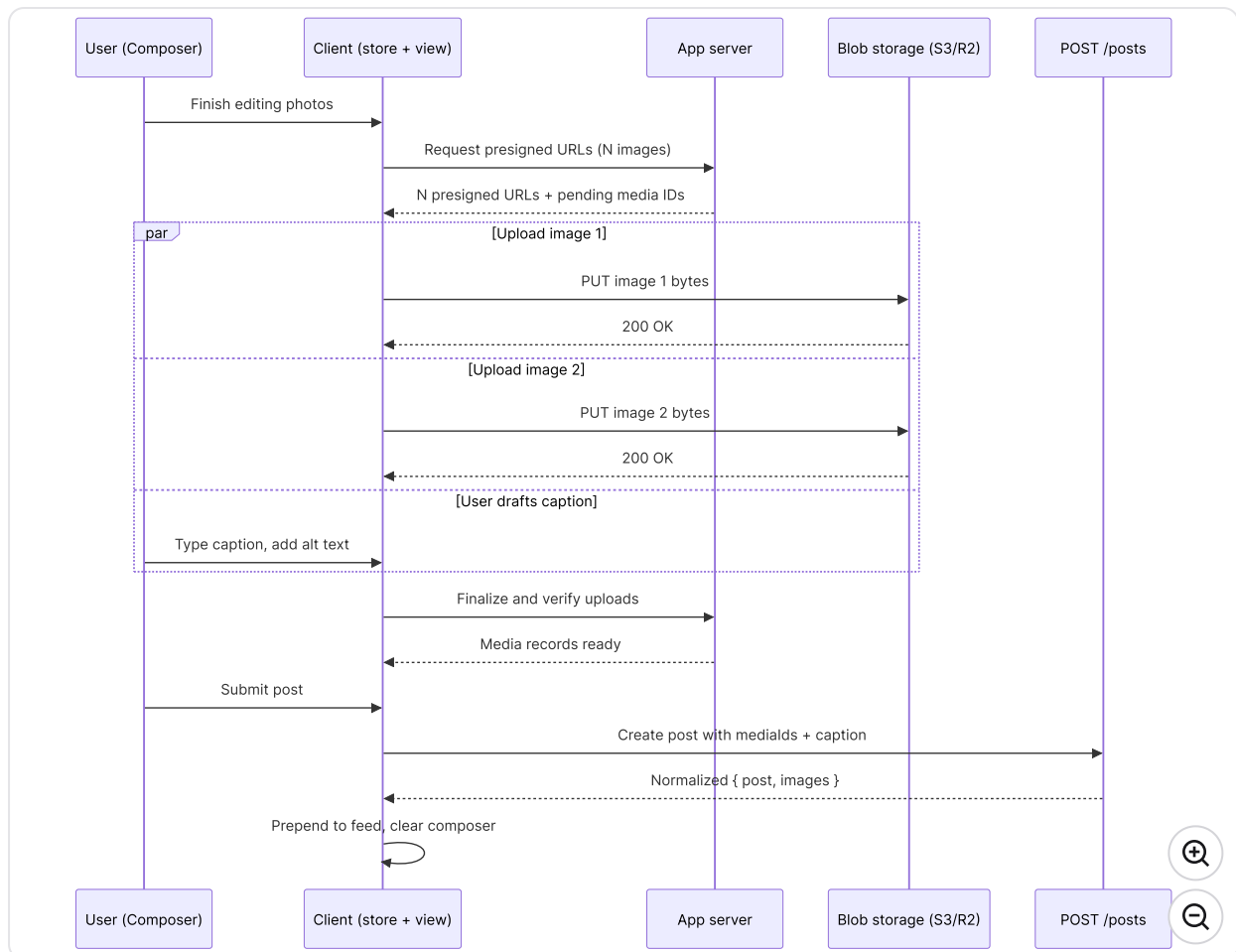
- **Reusable:** The image uploading API can be a general one that is used across the entire website and not just for the post creation flow (e.g., profile image upload).
- **Reduced bandwidth:** Practically, blob storage solutions like Amazon S3 and Cloudflare R2 are used for storing static assets like images. Clients can request a presigned URL and upload directly to the storage buckets without going through the app servers, which reduces bandwidth, leading to lower server costs.

✓ Upload photos via presigned URLs, then create the post by media ID

The client asks the server for a presigned URL, PUTs the image bytes straight to blob storage, and then calls the post creation endpoint with just the returned media IDs. This keeps large binaries off application servers, lets uploads run in parallel and in the background while the user is still writing the caption, and makes the final post creation a small JSON request.

The upload capability is short-lived and bound to the user, draft, object key, type, size, and checksum. The `ComposerDraft` owns each image's progress and retry state, requesting a replacement capability if one expires. Uploaded bytes remain non-public until the server verifies and re-encodes them, strips metadata, scans them, generates variants, and marks the media `ready`; `POST /posts` accepts only owned, ready media IDs.

For bounded photo sizes, a one-shot `PUT` is enough. If large files, unreliable mobile networks, or abandoned drafts become important, discuss resumable uploads, cancellation, and expiry-based orphan cleanup.



If the interviewer asks about retrying an ambiguous post-creation timeout, reuse one idempotency key for the same submission so the server can return the original result instead of creating a duplicate post.

Exercise 1 of 5 Beta

A composer uploads several images while the author continues editing a caption. Which API boundary best separates binary transfer from final post publication?

- 1 Send the caption and every base64-encoded image through one post-creation request after the author finishes editing.
- 2 Upload directly to blob storage without creating media records, then pass the resulting storage object keys in the post mutation.
- 3 Issue scoped upload targets with pending media IDs, upload bytes directly to blob storage, then create the post with the verified ready media IDs.



Reveal answer

Check answer ↩

Optimizations and deep dive

The deeper discussion builds on the News Feed optimizations but pays extra attention to image-heavy concerns, upload flow, and media accessibility:

- **Feed optimizations**: This section explains which News Feed performance and state-management techniques also apply to an Instagram-style feed.
- **Image carousel optimizations**: This section covers the carousel behavior used when a post contains multiple images.
- **Rendering images**: This section discusses image formats, sizing, CDN delivery, lazy loading, and responsive rendering.
- **Image editing**: This section explains in-browser cropping, resizing, filters, stickers, text overlays, and upload preparation.
- **Accessibility (a11y)**: This section covers text alternatives, keyboard interaction, and screen-reader support for an image-first product.

Feed optimizations

The optimizations covered in the **News Feed system design** also apply. Locally, keep rendering bounded, retain a focused virtualized post, and restore navigation by stable post anchor rather than only a pixel offset.

- Rendering approach
- Infinite scrolling implementation
- Virtualized lists
- Code splitting JavaScript
- Loading indicators
- Preserving feed scroll position
- Lazy load code
- Optimistic updates
- Timestamp rendering
- Icon rendering

Which practices keep a long photo feed responsive and correctly ordered? Select all that apply.

- ☐ 1 Continue with server-provided cursors so new insertions do not shift older-page boundaries.
- ☐ 2 Cache each page as an immutable nested snapshot and concatenate pages without reconciling overlapping post IDs.
- ☐ 3 Keep all visited posts mounted but pause offscreen media so reverse scrolling remains instant.
- ☐ 4 Bound mounted work while preserving focused or actively viewed posts.
- ☐ 5 Merge entities by ID and append only unseen feed references.

[Reveal answer](#)[Check answer ↩](#)

Image carousel optimizations

The images within a post are shown as a carousel, so the [Image Carousel system design article](#) is also useful for this question. Locally, load the active and adjacent slides, identify each slide by stable media ID, and expose a post-specific carousel label plus slide position such as “2 of 5”.

Exercise Beta

Which carousel-loading policy best balances fast post rendering and smooth next-slide navigation?

- ☐ 1 Preload and decode all carousel images at their largest available size before revealing the first slide.
- ☐ 2 Load only the active slide and begin the next request after navigation, using a skeleton during the transition.

3

Load the active responsive image, reserve its geometry, and prefetch only adjacent slides when budgets allow.



Reveal answer

Check answer ↩

Rendering images

- Use modern image formats such as WebP, which provides superior lossless and lossy image compression.
- `<img>` s should use proper `alt` text.
 - Prefer author-provided text. A generated fallback should be labeled and editable so the author can correct sensitive, speculative, or poorly localized descriptions.
- Image loading based on device screen properties
 - The server can use request signals such as Client Hints to choose a stable candidate family, provided those signals vary the cache key.
 - Return stable variants with intrinsic dimensions and use `srcset` / `sizes` so the browser makes the final choice for the rendered slot and device pixel ratio.
- Image caching and loading priority
 - Use versioned URLs for processed variants so browsers and CDNs can cache them without coupling the image bytes to caption or alt-text updates.
 - Load the likely first-viewport LCP image eagerly and lazy-load offscreen images.
- Adaptive image loading based on network speed
 - **Devices with good internet connectivity/on WiFi:** Prefetch offscreen images that are not in the viewport yet but are about to enter the viewport.
 - **Poor internet connection:** Render a low-resolution placeholder image and require users to explicitly click on it to load the high-resolution image.



Serve multiple sizes and modern formats, and let the browser choose

Produce WebP (or AVIF) variants at several widths during the upload pipeline and expose them via `srcset` plus `sizes`. The browser will then pick the right candidate for the current viewport and device pixel ratio, which matters far more in a photo-

first feed than in a text-heavy one because images dominate both byte budget and perceived load time.

Exercise 1 of 3 Beta

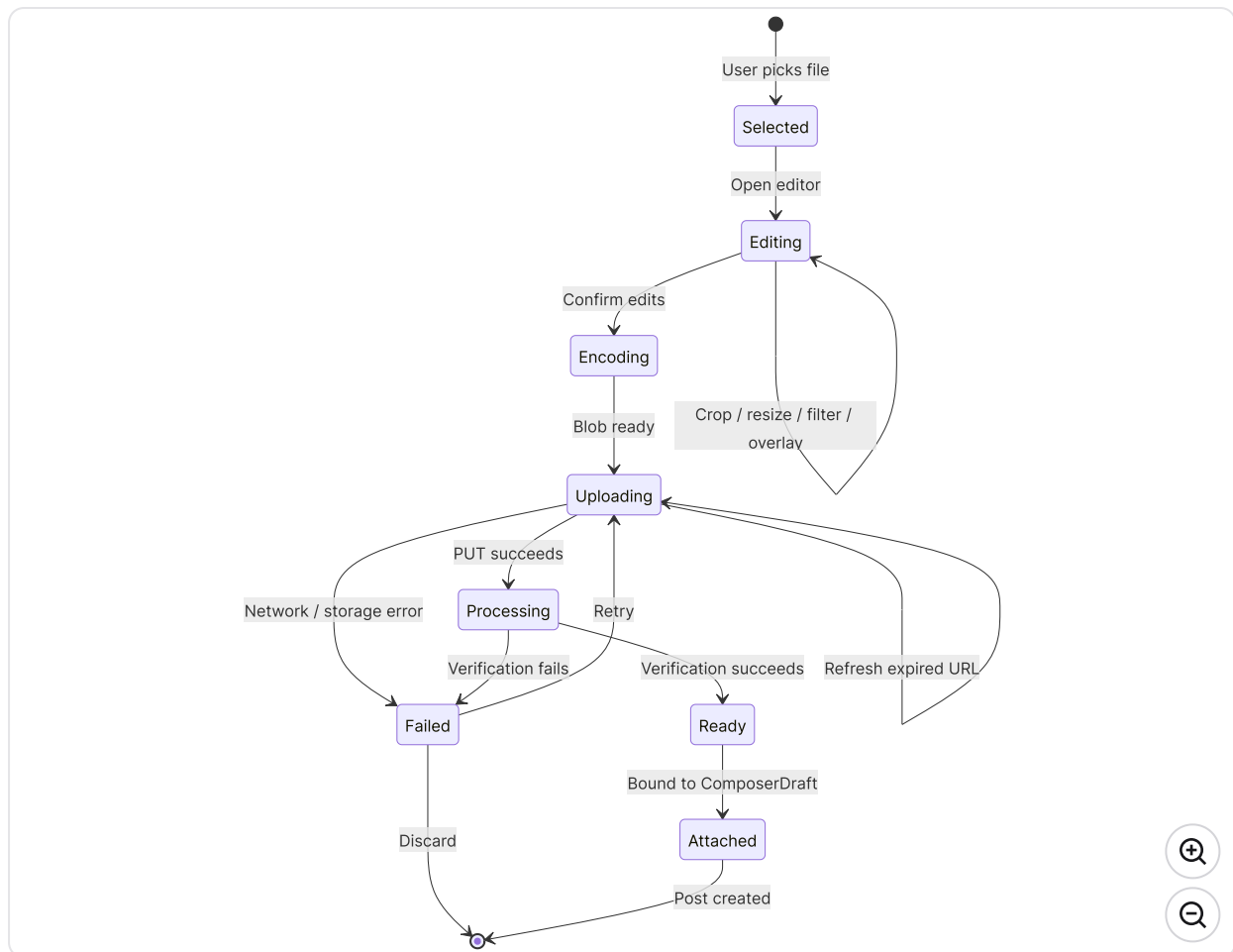
Which image-rendering choices improve visual stability and bandwidth use? Select all that apply.

- ☐ 1 Reserve width, height, or aspect ratio before loading.
- ☐ 2 Render cropped photos as CSS backgrounds and put the photo description on the surrounding card link.
- ☐ 3 Lazy-load below-the-fold feed images while prioritizing the likely LCP image.
- ☐ 4 Provide responsive sources and an accurate `sizes` hint.
- ☐ 5 Use one high-resolution source per photo so feed and detail placements can reuse the same cached URL.
- ☐ 6 Use a low-resolution or color placeholder inside the reserved media box while the selected source loads.

[Reveal answer](#)[Check answer ↩](#)

Image editing

In-browser editing covers three capabilities before upload: cropping and resizing, applying filters, and drawing overlays like stickers or text.



Per-image lifecycle from selection to attached media

Cropping and resizing

Cropping and resizing of the images can be done via HTML5's `<canvas>`.

```

1  const canvas = document.getElementById("image-editor");
2  const context = canvas.getContext("2d");
3  const file = fileInput.files[0];
4  const image = await createImageBitmap(file);
5  context.drawImage(image /* crop/resize parameters */);

```

Starting from the selected local `File` avoids a tainted cross-origin canvas and waits for decoding before drawing. Keep the source immutable, bound preview resolution, and move expensive transforms to a worker with `OffscreenCanvas` when profiling shows main-thread cost.

A lightweight editor can use inline transform and filter styles to preview changes before exporting the same parameters into the final bitmap.

Filters

CSS provides a `filter` property which allows applying filter functions like `blur`, `contrast`, `hue`, and `sepia`, to name a few. By using a combination of these filter functions, you can achieve Instagram-like filtering functionalities right within your browser.

Here are examples of the 1977 and Brannan filter effects, taken from [Instagram.css](#).

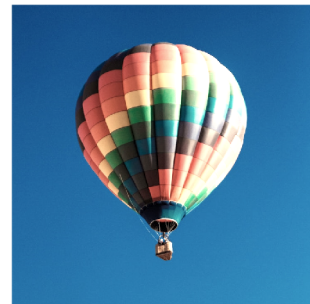
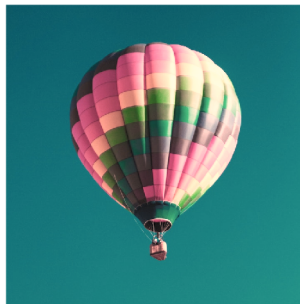
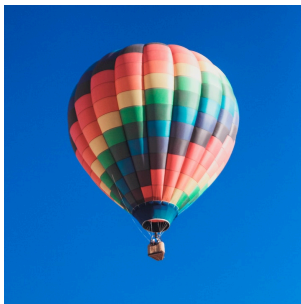
```
1 .filter-1977 {  
2   filter: sepia(0.5) hue-rotate(-30deg) saturate(1.4);  
3 }  
4  
5 .filter-brannan {  
6   filter: sepia(0.4) contrast(1.25) brightness(1.1) saturate(0.9)  
7     hue-rotate(-2deg);  
8 }
```

The following images are enhanced right within your browser using these CSS filters.

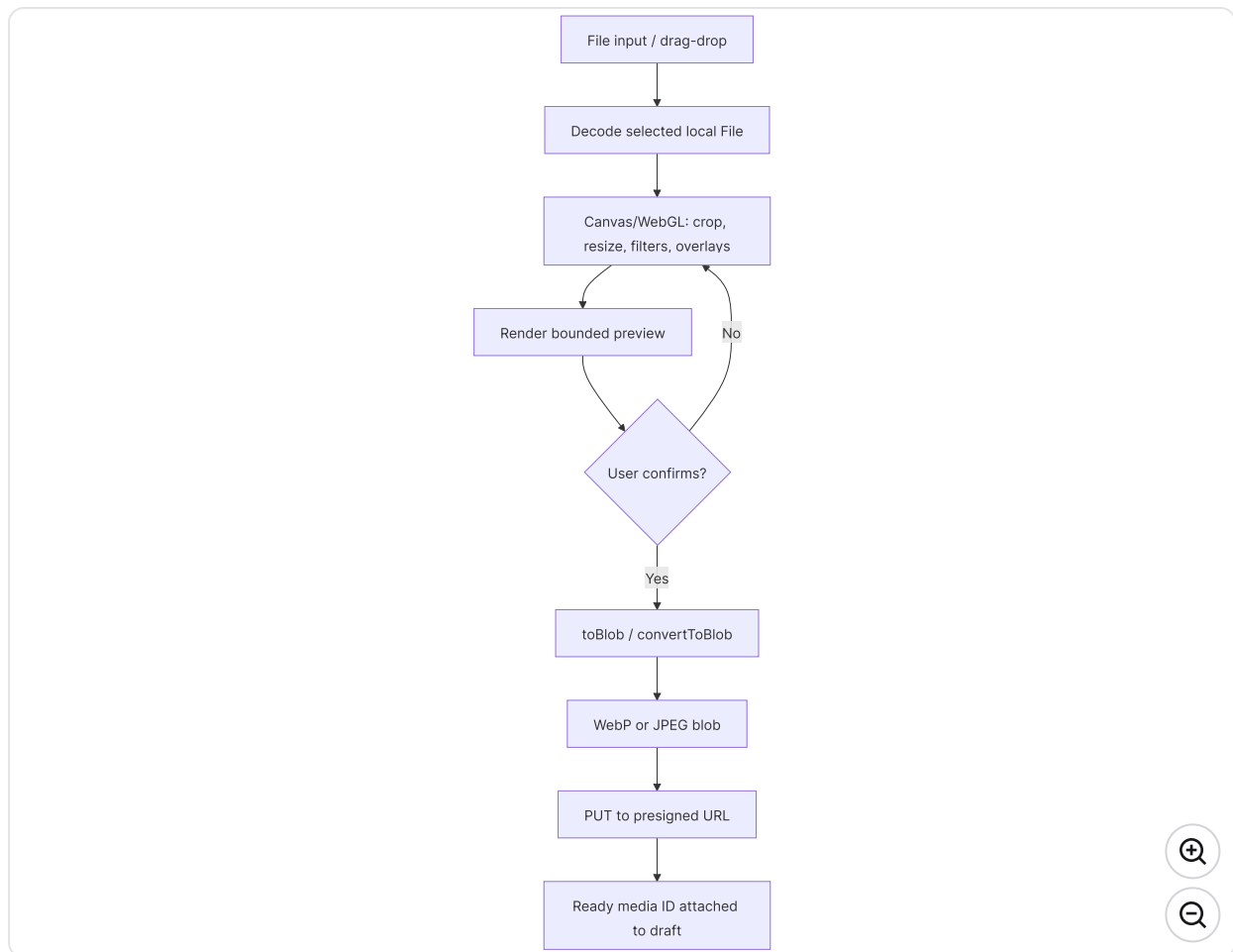
Original

1977

Brannan



The final upload must match the preview. Apply the same crop, rotation, overlays, and filter parameters in one origin-clean Canvas/WebGL pipeline, then export with `toBlob()` or `OffscreenCanvas.convertToBlob()`; a CSS-only preview is not itself an exported bitmap.



In-browser editing pipeline from file picker to uploaded blob

Exercise 1 of 2 Beta

Which practices keep basic client-side photo editing responsive during repeated adjustments? Select all that apply.

- ☐ 1 Send the full-resolution bitmap through a worker and block the preview on every input event.
- ☐ 2 Apply every input event directly to the full-resolution canvas so the preview always matches the eventual export exactly.
- ☐ 3 When profiling shows expensive pixel work, move it to a worker or GPU-backed path.
- ☐ 4 Use a bounded preview resolution and produce the final export only when needed.



Reveal answer

Check answer ←

Accessibility (a11y)

An image-first product leans heavily on text alternatives and keyboard operability, which are covered in the screen reader and keyboard interaction subsections below.

Screen readers

- `<img>` tags should have a meaningful `alt` description specified or use an empty string.
- Add `aria-label`s to the prev/next buttons within an image carousel.
- Give each carousel a post-specific label and a non-focusable `role="region"` or `role="group"` container with `aria-roledescription="carousel"`; expose each visible slide as a named group with its position and set size.
- Icon-only buttons should have the appropriate `aria-label`s if there are no accompanying labels (e.g., heart, share, bookmark).

Keyboard support

- Use the `<button>` HTML tag for the Prev/Next buttons where possible so that the buttons are focusable.
- Keep native Previous/Next buttons in the tab order. Do not add a redundant focus stop to the carousel wrapper unless the component deliberately defines its own composite keyboard interaction.

Exercise Beta

Which practices make a photo-sharing feed accessible? Select all that apply.

1

Make Like, Comment, Save, and composer actions reachable with visible focus.

☐

2

Expose author-provided or contextual alternative text for meaningful images.

☐

3

Label icon actions with hover and focus tooltips but omit programmatic accessible names.

☐

4

Use real labeled controls and expose slide position for multi-image posts.

☐

5

Send every reaction-count change from currently mounted cards through a polite live region.



Reveal answer

Check answer ←

Summary

Treat Instagram as News Feed plus Image Carousel plus a media delta. Once the inherited feed and carousel mechanics are named, focus on the Instagram-specific image pipeline, upload flow, in-browser editing, and media-heavy rendering concerns.

Most of the scaffolding comes from News Feed and Image Carousel. The Server, Controller, and normalized Client store shape the feed as in News Feed, with ordered post and media IDs rather than nested copies. Cursor-paginated requests drive a virtualized, infinite-scroll Feed UI so memory stays bounded as the user scrolls, while stable IDs and the feed generation keep page merges consistent. Each post hosts the Image Carousel patterns for multi-image navigation, with `srcset` and `sizes` picking the right asset per viewport and accessibility defaults such as image alternatives, labeled controls, and named slide positions.

The media pipeline is the Instagram-specific design. The Post composer asks the Server for a scoped upload capability per image, then PUTs a bounded number of images directly to blob storage while the user composes the caption. Post creation is a second call that references verified, ready media IDs, keeping the `ComposerDraft` submission small and the stored `Image` entities immutable. In-browser cropping, resizing, and filters use one Canvas/WebGL pipeline so the exported bitmap matches the preview. Every `Image` ships with `width` and `height` metadata so the Feed UI can reserve aspect-ratio boxes and avoid layout shift as posts hydrate. The upload, edit, and submit flow is distinctly Instagram's and deserves explicit walkthrough.

Spend interview time on the presigned-URL upload pipeline, the `<canvas>`-based editor, and the layout-stability contract around `width / height`; the inherited feed mechanics can be named quickly and moved past.

References

The references below group sources by topic, so it is easier to map each source back to the part of the article it informed.

Instagram engineering case studies

These references inform the overall client architecture, caching, and bundle-size choices discussed throughout the optimizations section.

- [Making Instagram.com faster: Part 1](#) for baseline performance work and the measurement approach behind the redesign.
- [Making Instagram.com faster: Part 2](#) for additional client-side loading and rendering strategies relevant to a photo feed.
- [Making Instagram.com faster: Part 3 — cache first](#) for the cache-first data access pattern referenced in the client caching discussion.
- [Making Instagram.com faster: Code size and execution optimizations \(Part 4\)](#) for code-splitting and execution tuning on a long-lived feed surface.

Accessibility

This reference informs the accessibility recommendations for the feed.

- [Crafting an accessible Instagram feed](#) for feed-specific accessibility patterns, focus management, and assistive technology considerations.
- [Launching Instagram Messaging on desktop](#) for how a large photo product layered real-time messaging onto an existing web client.

Image loading and performance APIs

These references back the image lazy-loading, infinite scroll, and rendering-strategy guidance covered in the optimizations section.

- [Intersection Observer API | MDN](#) for infinite scroll triggers and prefetch boundaries around the viewport.
- [Lazy loading images and iframe elements | web.dev](#) for browser-native image lazy loading guidance used in the media optimization discussion.

- [Rendering on the Web | web.dev](#) for the SSR, CSR, and hybrid rendering background that informs the rendering approach for feed and profile pages.
- [Instagram.css](#) for a community CSS reference that mirrors Instagram-style visual treatments when prototyping a photo feed.

Related system design articles

The articles below dive deeper into mechanics that are touched on but not fully explored here.

- [News Feed \(Facebook\)](#) for the shared feed architecture, pagination, and client store patterns that a photo feed builds on top of.
- [Pinterest](#) for masonry-style image layouts and additional photo-heavy performance considerations.

Exercises

Exercise 1 of 18 Beta

You are asked to design a front-end photo-sharing experience centered on browsing and creating image posts. Which requirements should be included before choosing the architecture? Select all that apply.

1 Use one caption for the overall post and allow filters to be applied before upload. ☐

2 Use infinite scrolling for continuation and design primarily for mobile while keeping desktop and tablet usable. ☐

3 Include comments, direct messages, follower-graph updates, and recommendation ranking in the baseline because each can affect photo discovery. ☐

4 Cover browsing the followed-photo feed, opening a public post, and creating a post by selecting, filtering, and captioning photos. ☐



Reveal answer

Check answer ←

Discussions

Normal T_T | B I U   <> Aa   </> 

Type comment here...

0/1000

Post

 Report an issue



✓ Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

- Get started
- JavaScript functions
- User interface coding
- System design
- Quiz

Guides

- Front End Interview Playbook
- Front End System Design Playbook

Study Plans

- 1 Week Plan
- 1 Month Plan
- 3 Months Plan

Company

- Pricing
- Promotions
- Roadmap
- About

[React Interview Playbook](#)

[Behavioral Interview Playbook](#)

[Team](#)

[Contact us](#)

[Become an affiliate](#)

[Careers](#)

[Blog](#)

[Medium](#)

[DEV Community](#)



English (US) ▾

© 2026 Codeney Pte Ltd. All rights reserved.

[Privacy Policy](#)

[Terms of Service](#)